

1)

[1, 2, 3, 6] and [-3, 0, 6, 7]

Compare 1 and -3 take -3
result array [-3]

Compare 1 and 0 take 0
[-3, 0]

Compare 1 and 6 take 1
[-3, 0, 1]

Compare 2 and 6 take 2
[-3, 0, 1, 2]

Compare 3 and 6 take 3
[-3, 0, 1, 2, 3]

Compare 6 and 6 take 6
[-3, 0, 1, 2, 3, 6]

Compare 6 and 7 take 6
[-3, 0, 1, 2, 3, 6, 6]

add 7

[-3, 0, 1, 2, 3, 6, 6, 7]

2) $[-21, 5, 7, -10, 61, 8, 3, 10]$

Comp 5 to before

$[-21, 5, 7, -10, 61, 8, 3, 10]$

Comp 7 to before

$[-21, 5, 7, -10, 61, 8, 3, 10]$

Comp -10 to before

$[-21, -10, 5, 7, 61, 8, 3, 10]$

Comp 61 to before

$[-21, -10, 5, 7, 61, 8, 3, 10]$

Comp 8 to before

$[-21, -10, 5, 7, 8, 61, 3, 10]$

Comp 3 to before

$[-21, -10, 3, 5, 7, 8, 61, 10]$

Comp 10 to before

$[-21, -10, 3, 5, 7, 8, 10, 61]$

3)

$$S_2 = [2, 619, 11, 5, 620, 4]$$

$$\text{left} = 0 \text{ and } S_2[\text{left}] = 2$$

$$\text{right} = 5 \text{ and } S_2[\text{right}] = 4$$

$$\text{center} = (\text{left} + \text{right})/2 = 2 \text{ and } S_2[\text{center}] = 11$$

$$\text{Pivot} = \text{median of } 2, 4, 11 = 4$$

same thing with i and j

$$[2, 619, 11, 5, 620, \boxed{4}]$$

$$[2, 619, 11, 5, 620, \boxed{4}]$$

$$[2, 619, 11, 5, 620, \boxed{4}]$$

i stops @ 619, j stops @ 2. Since i and j have crossed we switch pivot and i.

$$[2, \boxed{4}, 11, 5, 620, 619]$$

now we can make our second partition

$$S_1 = [2] \text{ pivot} = 4 \quad S_2 = [11, 5, 620, 619]$$

S_1 is sorted because it's one value but now sort S_2 again

$$S_2 = [11, 5, 620, 619]$$

$$\text{left} = 0 \text{ and } S_2[\text{left}] = 11$$

$$\text{right} = 3 \text{ and } S_2[\text{right}] = 619$$

$$\text{center} = (\text{left} + \text{right})/2 = 1 \text{ and } S_2[\text{center}] = 5$$

$$\text{Pivot} = \text{median of } 11, 5, 619 = 11$$

3.

3)

$A = [-5, 4, 2, 619, 11, 5, 620, -3]$

$\text{left} = 0$ and $A[\text{left}] = -5$

$\text{right} = 7$ and $A[\text{right}] = -3$

$\text{center} = (\text{left} + \text{right}) / 2 = 3$ and $A[\text{center}] = 619$

Pivot = median of $-5, -3, 619 = -3$

$i = \text{looking for bigger than pivot}$

$j = \text{looking for smaller than pivot}$

$[-5, 4, 2, 619, 11, 5, 620, -3]$

$[-5, 4, 2, 619, 11, 5, 620, -3]$

$[-5, 4, 2, 619, 11, 5, 620, -3]$

i stops @ 4

j stops @ -5

$[-5, -3, 2, 619, 11, 5, 620, 4]$

since j and i have crossed we switch pivot and
 now we can do our first partition

$S_1 = [-5]$ pivot = -3 $S_2 = [2, 619, 11, 5, 620, 4]$

S_1 is sorted because it's one value but now
 we sort S_2

$\triangleright$ 3 $[11, 5, 620, 619]$
 $\triangleright$ we need to move the pivot to the end
 $\triangleright$ so swap 11 and 619
 $\triangleright$ $[619, 5, 620, 11]$
 $\triangleright$ pivot

$\triangleright$ $[619, 5, 620, 11]$

$\triangleright$ $[619, 5, 620, 11]$

$\triangleright$ Since i is bigger than pivot and j is smaller than

$\triangleright$ pivot swap 5 and 619

$\triangleright$ $[5, 619, 620, 11]$

$\triangleright$ $[5, 619, 620, 11]$

$\triangleright$ Since i and j have crossed with can switch pivot

$\triangleright$ $[5, 11, 620, 619]$

$\triangleright$ Now we can make another partition

$\triangleright$ $S_1 = [5]$ pivot = 11 $S_2 = [620, 619]$

$\triangleright$ S_1 is sorted because its one value

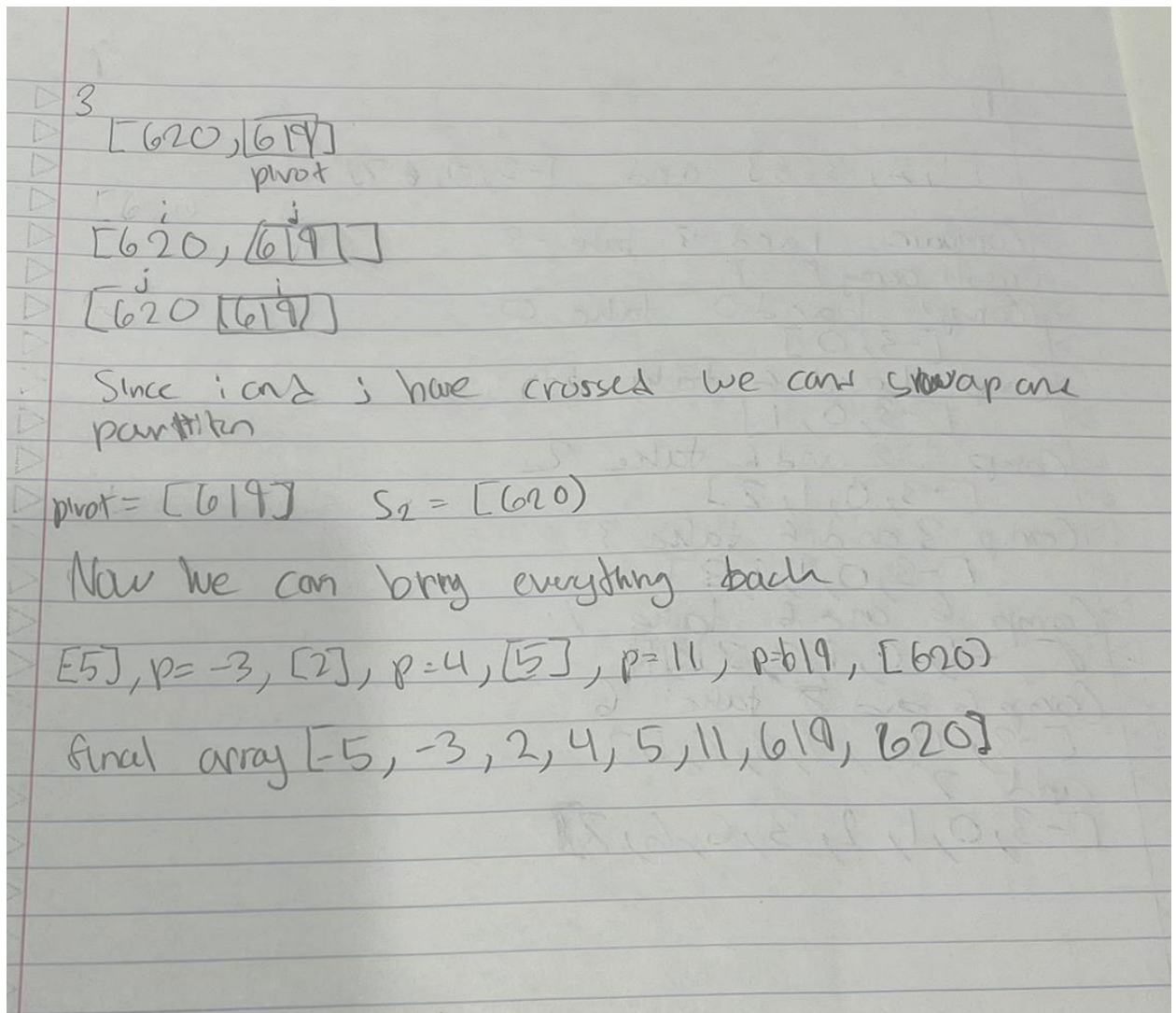
$\triangleright$ $S_2 = [620, 619]$

$\triangleright$ left = 0 and $S_2[\text{left}] = 620$

$\triangleright$ right = 1 and $S_2[\text{right}] = 619$

$\triangleright$ center = $(\text{left} + \text{right}) / 2 = 0$ and $S_2[\text{center}] = 619$

$\triangleright$ Pivot = 619, 619, 620 = 619



4.

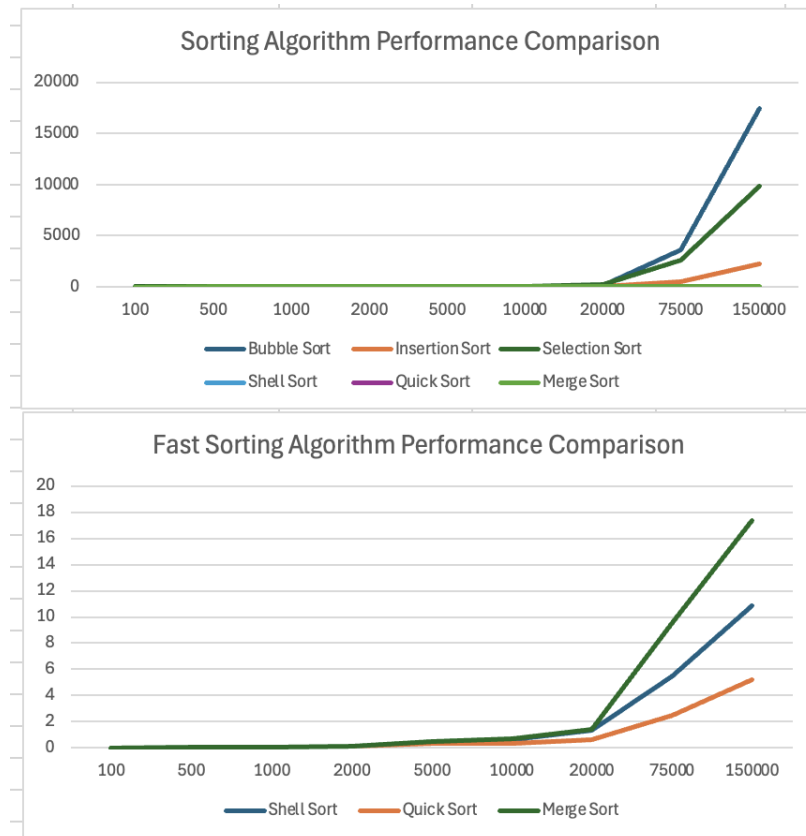
5. My personal ranking of the sorting algorithms are Quicksort, Mergesort, Shell sort, and finally a tie between Selection, Bubble, and Insertion. The reason I think Quick Sort is first is it literally has quick in the name. Second, I believe that the methods that use recursion are quicker because they divide and conquer, sorting the array in smaller pieces which makes the process faster. In the best and average case, the time complexity is $O(n \log n)$, which is the second fastest growth rate next to constant time. Then I would say Merge Sort because of recursion again and it has the same time complexity at $O(n \log n)$. Merge Sort is always $O(n \log n)$ even in the worst case, which makes it very reliable. Then I would say shell sort with a time complexity of $O(n^{3/2})$. Shell Sort compares elements that are far apart first, then gradually reduces the gap. Finally, I would rank Selection Sort, Bubble Sort, and

Insertion Sort in a tie for last place. All three have a time complexity of $O(n^2)$ because they use nested loops.

6. Code

7. Code

8. Code



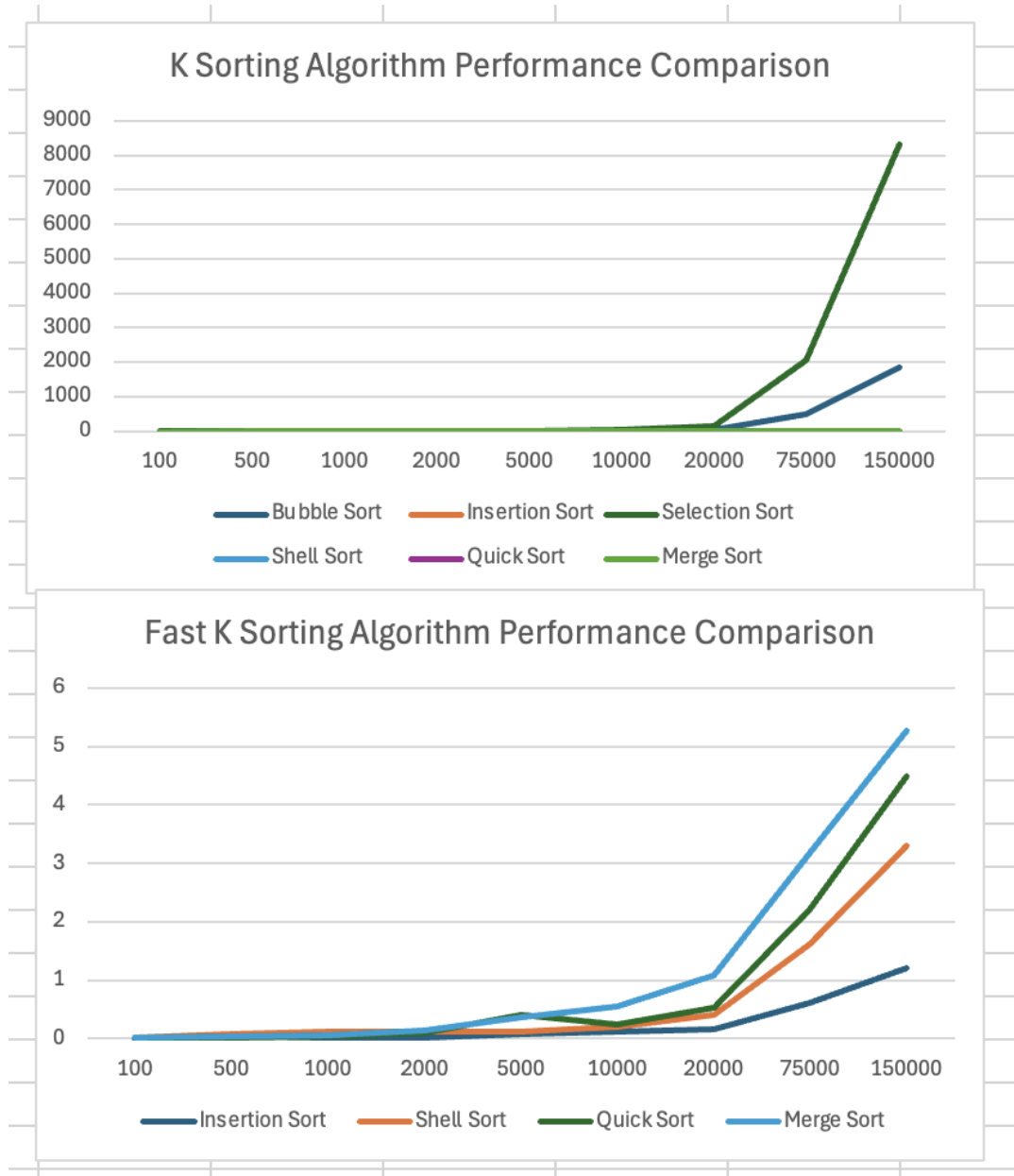
9.

The issue is that Bubble Sort and Selection Sort have times in the thousands of milliseconds while Quick Sort and Merge Sort have times around 10 ms. This difference makes the fast algorithms appear as flat lines near zero on this scale, making it impossible to compare their relative performance. To solve this I created a second graph to proper compare Shell, Quick, and Merge.

10. Through my experiments, I noticed that Bubble Sort and Selection Sort both took a couple of minutes to run at the higher array sizes. Insertion Sort was longer but went faster than the other $O(n^2)$ time complexity sorts. All the algorithms under an array size of 2,000 fared pretty equally. The differences started to become apparent after that. My predictions were basically confirmed other than Insertion Sort outperforming Bubble Sort and Selection Sort. Another thing I noticed when I ran this multiple times: the $O(n^2)$ algorithms had more varied times at larger array sizes, which means the individual order of each number in the array can affect the times greatly.

11. Code

12.



After K sorting at 10 I noticed that Bubble Sort and Selection Sort both went way faster than before. However who stole the show was insertion sort. At the largest array it got 1.2 ms which really shocked me. I believe it was because there was a higher chance of swapping numbers into the correct place instantly. The new ranking after k sort was Insertion, Shell, Quick, merge, bubble and finally selection. Did the same graph situation as number 9.